

Spark Interview: 2-Minute Debugging Reference

Fast recall for slow-job debugging, joins, shuffle, spill, skew, partitions, AQE, and infra failures

1. The 2-minute answer: slow job investigation flow

Scenario: a job normally processes ~5–10 TB in ~30 minutes, but today takes ~4 hours.

Question	Check	Likely direction
1. Where did the extra time go?	Spark UI → Jobs/Stages → compare normal vs slow run	Find the stage(s) responsible
2. One stage slow or everything slow?	Stage duration pattern	One stage → join/skew/spill; all stages → volume/infra/I/O
3. Few tasks slow or all tasks slow?	Task duration: median vs max	Few → skew; all → broad pressure/partition sizing
4. Is it a join?	SQL tab / physical plan / DAG	Identify BroadcastHash / ShuffledHash / SortMerge
5. Shuffle huge?	Shuffle Read + Shuffle Write	Join/aggregate/repartition cost
6. Spill huge?	Memory Spill + Disk Spill	Working set too large; sort/hash pressure
7. Executors unstable?	Executors tab, lost executors, retries, GC	Spot loss, OOM, recomputation, resource pressure

Interview opening script

“I would first compare the abnormal run with a normal run and isolate where the extra time is spent. In Spark UI I would identify the slow stage, then compare task durations and shuffle/spill metrics. If only a few tasks are outliers I suspect skew; if all tasks are slow I look at data volume, partition sizing, memory/I/O, or a changed plan. If the stage is a join, I verify the physical join strategy before changing hints or configuration. I also check executor loss/retries for infrastructure issues.”

The three patterns to remember

Pattern	What you see	First suspicion
A. One stage is much slower	One stage consumes most runtime	Join / skew / shuffle / spill / partition issue
B. Most stages are slower	Broad slowdown across job	More data, historical/backfill load, I/O, resource pressure, plan change
C. Executors/nodes unstable	Lost executors, retries, recomputation	Spot/preemptible loss, OOM, cluster/autoscaling issue

2. Spark UI: exactly what to inspect

UI location	Metric / signal	Interpretation
Stages	Stage duration	Find culprit stage(s)
Stage Details	Task Duration: median vs max	Large max/median gap → straggler/skew
Stage Details	Shuffle Read / Write	Large values → expensive repartition/join/aggregate
Stage Details	Disk Spill	High spill → task working set does not fit comfortably in memory
Stage Details	Memory Spill	Shows data spilled from in-memory structures; use with disk spill and duration
Stage Details	Input Size / Records	One task much larger → uneven partition / skew
Stage Details	GC Time	High percentage → memory pressure / object churn
Executors	Lost executors / failed tasks	OOM, spot loss, node failure, instability
SQL / explain	Physical operator	Know which join Spark actually chose

Fast diagnostic patterns

Median task = 30 sec; Max task = 3 hr

→ Few stragglers. Suspect skew / hot keys.

Most tasks = 20–30 min; most spill heavily

→ Broad issue. Suspect too-large partitions, insufficient resources, more data, or I/O.

Shuffle Read is multi-TB + SortMergeJoin

→ Expensive shuffle/sort. Check whether one side could broadcast or whether partition count is too low.

3. Join strategy cheat sheet

Join	Use when	Cost / risk	How to identify
BroadcastHashJoin	One side is genuinely small enough to replicate	Avoids large-side shuffle; risk = executor memory / OOM if forced too large	explain("formatted") → BroadcastHashJoin
ShuffledHashJoin	One side is smaller, but not broadcast-small; per-partition build side fits memory	Shuffles both sides; hash build can create memory pressure	explain → ShuffledHashJoin + BuildLeft/BuildRight
SortMergeJoin	Both sides are large	Shuffles + sorts both sides; robust at scale but can spill heavily	explain → SortMergeJoin

Mental model: tiny side → BroadcastHashJoin | moderately smaller side → ShuffledHashJoin | both sides large → SortMergeJoin.

Broadcast debugging

- If the slow stage has huge shuffle and one join side is small, check whether the normal run used BroadcastHashJoin but the slow run changed to SortMergeJoin.
- Check actual bytes, not just row count. Wide rows can make a “small” row-count table large.
- Possible reasons broadcast stopped: stale/missing statistics, size crossed threshold, changed filters/projection, changed plan/AQE behavior.
- Do not force broadcast blindly; broadcasting a large table can cause GC, OOM, executor loss.

4. Shuffle, spill, partitions, and skew

Symptom	Meaning	Typical action
Huge shuffle; little spill	Network/repartition cost is high	Revisit join strategy, filters, unnecessary repartition, pre-aggregation
Huge shuffle + huge spill on most tasks	Working set too large per task	Increase shuffle partitions; inspect memory/data volume; reduce data early
Only a few tasks have huge shuffle/spill	Uneven partitions / hot keys	Prove skew; inspect join-key frequency; AQE/skew handling/salting
High GC across most tasks	Memory pressure / object churn	Smaller partitions, memory sizing, serialization/data-shape review
Many tiny tasks	Over-partitioning / scheduler overhead	Reduce partitions / coalesce where appropriate

Skew: prove it before fixing it

Signal: most tasks look normal, but a few have dramatically larger Shuffle Read/Input Size and much longer duration.

```
fact.groupBy("customer_id")
  .count()
  .orderBy(F.desc("count"))
  .show(20)
```

- Look for hot keys (one/few keys with extreme counts), NULL-heavy keys, or sentinel values such as -1 / UNKNOWN.
- First choice: filter or handle meaningless/null keys separately if business logic permits.
- Then consider AQE skew splitting. Use manual salting only when needed.

Salting: 20-second reminder

Large/skewed side:

add salt bucket 0..N-1 to hot-key rows

Small side:

duplicate matching hot-key row across salt values

Join on: original_key + salt

Purpose: one hot key no longer owns one giant partition; its rows are distributed across multiple partitions/tasks.

Partition sizing

- If most tasks spill and each task handles very large shuffle blocks, increase `spark.sql.shuffle.partitions` so each task processes less data.
- Do not assume “more executor memory” is the first fix. Large partitions can stay pathological even on bigger executors.
- Too many partitions also hurts: many tiny tasks create scheduling overhead. Target balanced, reasonably sized tasks, not a magic number.

5. Infrastructure / executor failure branch

Signal	What may be happening	What to inspect
Executors disappear	Spot/preemptible termination, node failure, OOM	Executors tab, cluster events/logs
Tasks repeatedly retry	Executor loss or task failure	Failed task reason, executor stderr/logs
Previously completed work repeats	Lost intermediate/shuffle data or stage recomputation	Stage attempts, executor-loss timing
High GC then executor loss	Memory pressure / OOM	GC time, heap sizing, partition sizes, join strategy
Dynamic allocation churn	Executors added/removed aggressively	Allocation settings, idle timeouts, workload shape

6. Fix matrix: symptom → likely cause → action

Symptom	Likely cause	Fixes to consider
One/few tasks 100× slower	Skew / hot key	Filter special keys; AQE skew handling; salt hot keys
All tasks spill heavily	Partitions too large / memory pressure	Increase shuffle partitions; reduce data early; validate memory sizing
Multi-TB shuffle for join	Wrong/expensive join strategy	Broadcast small side; consider shuffled hash; pre-filter/project
Normal run broadcast, slow run sort-merge	Plan/statistics change	Compare physical plans; refresh stats; validate threshold; hint only if safe
High GC / OOM	Too-large working set / bad broadcast / large hash table	Smaller partitions; avoid oversized broadcast; memory tuning
Most stages suddenly slower	Input volume spike / backfill / storage slowdown	Compare bytes/records; upstream ingestion; storage metrics
Lost executors / retries	Spot/preemptible loss or infra issue	Cluster events; retry/recompute impact; node type strategy
Many tiny tasks	Over-partitioned workload	Coalesce/reduce partitions where safe

7. Commands / snippets worth remembering

```
df.explain("formatted")
```

```
# Look for: BroadcastHashJoin / ShuffledHashJoin / SortMergeJoin
```

```
spark.conf.set("spark.sql.shuffle.partitions", <appropriate value>)
```

```
# Use when tasks are too large; validate by observing lower spill and balanced durations.
```

```

from pyspark.sql.functions import broadcast
result = fact.join(broadcast(dim), "key")
# Only when the broadcast side is truly safe to replicate.

df.groupBy("join_key").count().orderBy(F.desc("count")).show(20)
# Prove skew / hot keys.

```

8. Interview traps to avoid

- Do not jump straight to “increase memory.” Diagnose first.
- Do not jump straight to “broadcast.” Prove the join is expensive and the broadcast side is safe.
- Do not say “skew” only because the job is slow. Show the evidence: max task duration and partition/shuffle size far above median.
- Do not describe spot loss as simply “work shifts to another executor.” Say tasks may retry elsewhere and lost intermediate/shuffle data may need recomputation.
- Do not assume the configured join strategy is the executed one. Verify the physical plan, especially with AQE.

9. Final 30-second checklist

Ask yourself	Yes →
One stage dominates runtime?	Drill into that stage
Few tasks are extreme outliers?	Skew / hot keys
All tasks spill?	Partition sizing / memory / data volume
Huge shuffle on join?	Inspect physical join strategy
One side tiny?	Broadcast candidate
One side moderately smaller?	ShuffledHash candidate if per-partition build side fits
Both sides huge?	SortMerge is expected; optimize shuffle/partitions/skew
Executors lost/retrying?	Infrastructure / spot / OOM branch
All stages slower?	Data volume / source / I/O / broad resource issue